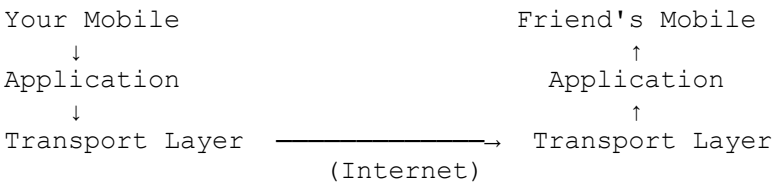
UNIT-4

1. Transport Layer

The **Transport Layer** is the **4th layer of the OSI model**. It provides **end-to-end communication between applications running on different devices**.

When you send a WhatsApp message:



The Transport Layer makes sure the data reaches the **correct application**.

Main functions

1. **Process-to-process delivery** – delivers data to the correct application.
2. **Segmentation** – breaks large data into smaller segments.
3. **Reassembly** – combines segments at the destination.
4. **Flow control** – controls the speed of data transmission.
5. **Error control** – handles lost or damaged data.
6. **Congestion control** – helps reduce network congestion.
7. **Port addressing** – identifies the correct application using port numbers.

Main protocols

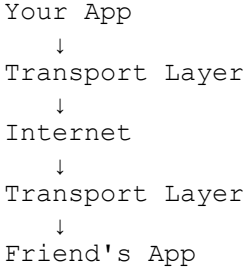
- **TCP** – reliable and connection-oriented.
- **UDP** – faster and connectionless.

2. Transport Services

The **Transport Service** is the service provided by the **Transport Layer** to enable communication between **applications running on different devices**.

Example

Suppose you send a message from your mobile to your friend's mobile:



The Transport Layer makes sure the data is delivered to the **correct application**.

Transport Services

1. Process-to-Process Delivery
Delivers data from a specific application on one computer to the correct application on another computer.

Example:
You send a message → it reaches the **WhatsApp application** on your friend's phone.

2. Reliable Data Transfer
Ensures data reaches correctly and in the proper order.

Example:
TCP can retransmit data if a segment is lost.

3. Flow Control
Controls the speed of data transmission so that the receiver is not overloaded.

Example:
A fast computer sends data at a speed the receiving computer can handle.

4. Error Control
Detects lost or damaged data and helps recover it.

Example:
If a TCP segment is lost, TCP can request/retransmit it.

5. Connection Management
Provides connection establishment and termination when using TCP.

```
Connection → Data Transfer → Connection Closed
```

6. Multiplexing and Demultiplexing
Allows multiple applications to use the network simultaneously and delivers incoming data to the correct application using **port numbers**.

Example

Suppose your computer is using:

```
Browser  → Port 443
Email    → Port 25
Another App → Port 5000
```

The Transport Layer uses the **port number** to deliver data to the correct application.

3. Services provided to Upper Layer

The **Transport Layer provides communication services to the upper layers**, mainly the **Application Layer**.

```
Application Layer
       ↑
   Services
       ↑
Transport Layer
       ↑
   Network Layer
```

Main services:

1. Process-to-Process Delivery
Delivers data from a specific application on one computer to the correct application on another computer.

Example:
You send a message → it reaches the **WhatsApp application** on your friend's phone.

2. Reliable Data Transfer
Ensures data reaches correctly and in the proper order.

Example:
TCP can retransmit data if a segment is lost.

3. Flow Control
Controls the speed of data transmission so that the receiver is not overloaded.

Example:
A fast computer sends data at a speed the receiving computer can handle.

4. Error Control
Detects lost or damaged data and helps recover it.

Example:
If a TCP segment is lost, TCP can request/retransmit it.

5. Connection Management
Provides connection establishment and termination when using TCP.

```
Connection → Data Transfer → Connection Closed
```

6. Multiplexing and Demultiplexing
Allows multiple applications to use the network simultaneously and delivers incoming data to the correct application using **port numbers**.

Example

Suppose your computer is using:

```
Browser  → Port 443
Email    → Port 25
Another App → Port 5000
```

The Transport Layer uses the **port number** to deliver data to the correct application.

4. Transport Service Primitives

Transport Service Primitives are the operations offered by the Transport Layer to applications for communication.

The five basic primitives are:

1. LISTEN – Waits for an incoming connection.
2. CONNECT – Establishes a connection.
3. SEND – Sends data.
4. RECEIVE – Receives data.
5. DISCONNECT – Terminates the connection.

Consider WhatsApp communication:

1. Establish a connection with another user.
2. Send a message.
3. Receive a reply.
4. Close the communication.

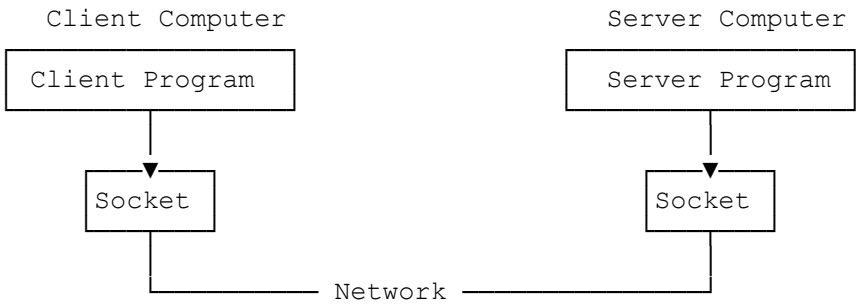
Similarly, the Transport Layer uses service primitives to perform these operations.

5. Berkeley Sockets

Berkeley Sockets, also called **BSD Sockets**, are a set of programming functions/API used to enable **communication between applications over a computer network**.

A socket is an endpoint of communication between two applications.

For example, when a client application communicates with a server application, sockets provide the mechanism for sending and receiving data.



It provides functions such as:

socket()

bind()
listen()
accept()
connect()
send()
recv()
close()

1. socket()

Creates a socket.

```
socket()
   ↓
Communication endpoint created
```

2. bind()

Assigns an **IP address and port number** to the server socket.

Example:

```
IP Address = 192.168.1.10
Port       = 5000
```

3. listen()

Server waits for clients.

```
Server
   ↓
listen()
   ↓
Waiting for client...
```

4. accept()

Server accepts the client's connection.

```
Client ——— Connection Request ——→ Server
                                    |
                                 accept()
                                    |
                                    ▼
                                 Connected
```

5. connect()

The client requests a connection to the server.

```
Client ——— connect() ———→ Server
```

6. send()

Sends data.

```
Client —— "Hello" ——→ Server
```

7. recv()

Receives data.

```
Client ◄—— "Hi" ——— Server
```

8. close()

Ends the communication.

6. Internet File Server

An **Internet File Server** is a computer/server that **stores files and allows users to upload, download, or access those files through the Internet**.

Example

Suppose a college has a server containing:

```
🖿 College File Server
   ├— 🗎 Notes.pdf
   ├— 🗎 QuestionPaper.pdf
   ├— 🗎 Results.xlsx
   └— 🖿 Assignments
```

Students can access these files through the network.,here students can

- **Upload** files → send files to the server
- **Download** files → get files from the server
- **View/Access** files → use files stored on the server

- **Share** files → allow other authorized users to access them

7. Elements of transport protocols

The **Transport Protocol** is responsible for providing communication between applications running on different computers.

The important **elements of transport protocols** are:

1. Addressing

The Transport Layer uses **port numbers** to identify applications.

```
IP Address  → Which computer?
Port Number → Which application?
```

Example:

```
192.168.1.10 : 5000
      |          |
   Computer    Application
```

2. Connection Establishment

Before sending data, TCP establishes a connection between sender and receiver.

```
Client ——— Connection Request ———▶ Server
Client ◀——— Connection Accepted ——— Server
```

3. Connection Release

After communication is completed, the connection is closed.

```
Client ——— Finish ———▶ Server
Client ◀——— Close ——— Server
```

4. Flow Control

Flow control prevents a **fast sender from sending more data than the receiver can handle**.

Flow control controls the amount of data being sent.

5. Error Control

Error control detects problems such as:

- Lost data
- Damaged data
- Duplicate data
- Data arriving in the wrong order

TCP can retransmit lost data.

6. Multiplexing and De multiplexing

Allows multiple applications to use the network simultaneously and delivers incoming data to the correct application using **port numbers**.

7. Congestion Control

Congestion occurs when there is **too much traffic in the network**.

Congestion control reduces the sending rate when the network becomes overloaded.

8. Addressing

Addressing is the process of identifying the sender and receiver in a computer network so that data reaches the correct destination.

Types of Addressing in Computer Networks

There are four main types of addresses:

1. Physical Address (MAC Address)

Identifies a device within a local network.

Data Link Layer

Example: 00:1A:2B:3C:4D:5E

2. Logical Address (IP Address)

Identifies a device across different networks.

Network Layer

Example: 192.168.1.10

3. Port Address

Identifies the particular application or process.

Transport Layer

Example: HTTP – Port 80

4. Specific Address

Identifies a particular application or resource.

Application Layer

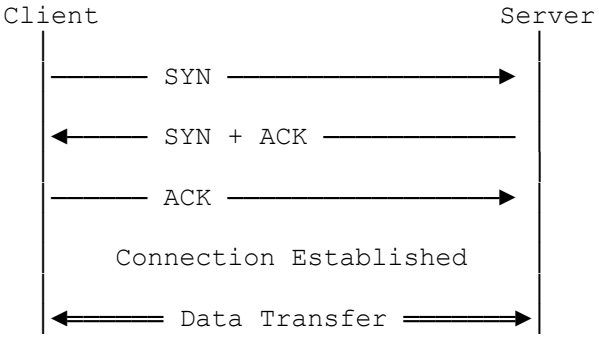
Example: www.google.com, email address

9. Connection establishment

TCP is a **connection-oriented protocol**, so before sending actual data, it establishes a connection between the client and server.

TCP uses a process called the **Three-Way Handshake**.

Three-Way Handshake



Step 1: SYN

The **client sends a SYN segment** to the server.

SYN = Synchronize

It requests:

"Can we establish a TCP connection?"

It also contains the client's initial sequence number.

Step 2: SYN + ACK

The server receives the SYN and sends back **SYN + ACK**.

- **SYN** → Server also wants to establish the connection.
- **ACK** → Server acknowledges the client's SYN.

Meaning:

"I received your request, and I am ready to communicate."

Step 3: ACK

The client sends an **ACK** back to the server.

Meaning:

"I received your response."

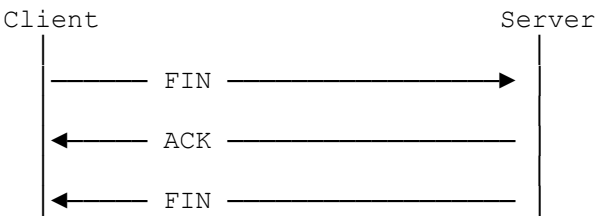
Now the TCP connection is established.

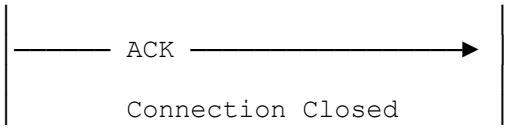
10. Connection Release

TCP connection release is the process of **properly closing a TCP connection after data communication is completed**.

TCP normally uses a **four-way handshake** to terminate a connection.

Four-Way Handshake





Step 1: FIN

The client sends a **FIN** segment to the server.

FIN = Finish

Meaning:

"I have finished sending data."

Step 2: ACK

The server sends an **ACK** to acknowledge the client's FIN.

Meaning:

"I received your request to close."

The server can still send remaining data if necessary.

Step 3: FIN

After the server finishes sending its data, it sends its own **FIN**.

Meaning:

"I have also finished sending data."

Step 4: ACK

The client sends the final **ACK**.

Meaning:

"I received your FIN."

The TCP connection is then closed.

11. Error Control and Flow Control

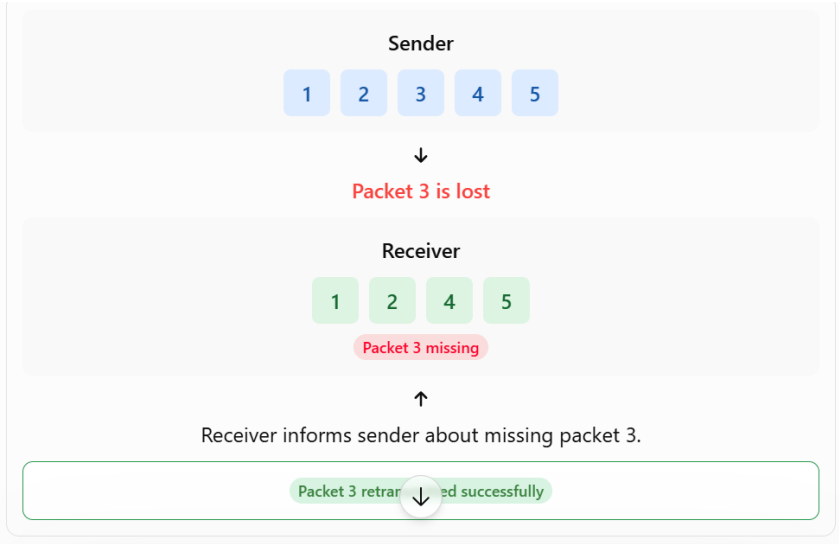
Error Control and Flow Control are two important functions of the Transport Layer. They help in reliable and proper data transmission between sender and receiver.

1. Error Control

Error Control is the process of detecting and correcting errors or recovering lost data during transmission.

Working:

- Sender sends data to the receiver.
- Receiver checks whether the data is received correctly.
- If data is lost or damaged, the receiver requests retransmission (or the sender detects a missing acknowledgment).
- Sender sends the data again.



Techniques of Error Control

- Error detection
- Acknowledgment (ACK)
- Negative acknowledgment (NAK)
- Retransmission
- Sequence numbers
- Checksum

2. Flow Control

Flow Control is the technique of controlling the speed of data transmission so that the sender does not send data faster than the receiver can handle.

Working:

- Sender sends data to the receiver.

- Receiver processes the incoming data.
- If the receiver is busy, it asks the sender to slow down or wait.
- Sender adjusts its sending rate.

Example

Imagine a teacher giving notes to students.

- Teacher = Sender
- Students = Receiver
- Notes = Data

If the teacher gives notes too quickly, students cannot write properly.

So, the teacher slows down and gives students enough time.

This is called Flow Control.

Techniques of Flow Control

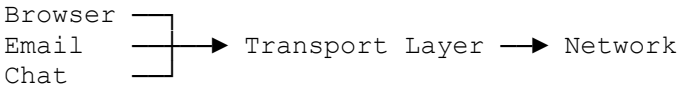
1. Stop-and-Wait
2. Sliding Window

12. Multiplexing and Demultiplexing

These are functions of the **Transport Layer** that help multiple applications communicate over the network.

1. Multiplexing

Multiplexing means collecting data from multiple applications and sending it through the network.



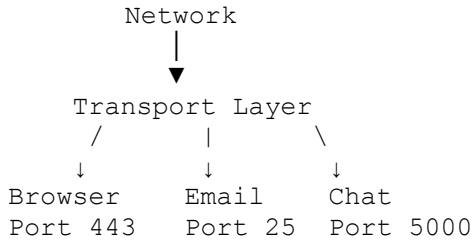
The Transport Layer adds **port numbers** so the data can be identified later.

Example

```
Browser → Port 443
Email   → Port 25
Chat    → Port 5000
```

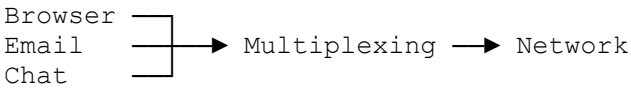
2. Demultiplexing

Demultiplexing means receiving data from the network and delivering it to the correct application.

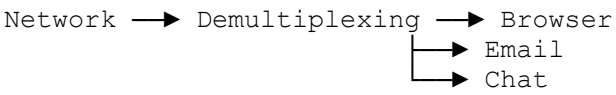


The Transport Layer checks the **port number** and delivers the data to the correct application.

At the sender:



At the receiver:

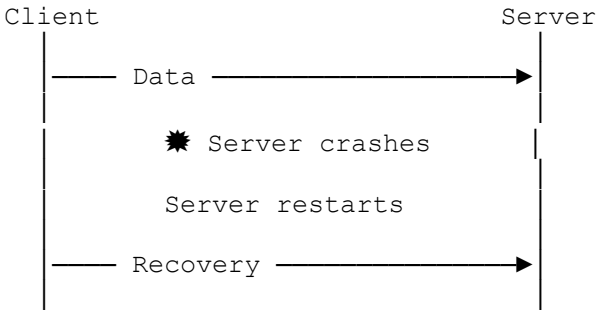


13. Crash Recovery

Crash recovery means recovering communication after a computer, server, or communication process suddenly crashes and restarts.

Example

Suppose a client and server are communicating:



After the crash, the system needs to determine:

- Was the previous data received?
- Which data needs to be sent again?
- Is the old connection still valid?
- Should a new connection be established?

Working

Transport protocols can use mechanisms such as:

1. **Sequence numbers** – identify data segments.
2. **Acknowledgements (ACKs)** – confirm received data.
3. **Timeouts** – detect when a response is not received.
4. **Retransmission** – send lost data again.
5. **Re-establishing the connection** – create a new connection if necessary.

14. Congestion Control

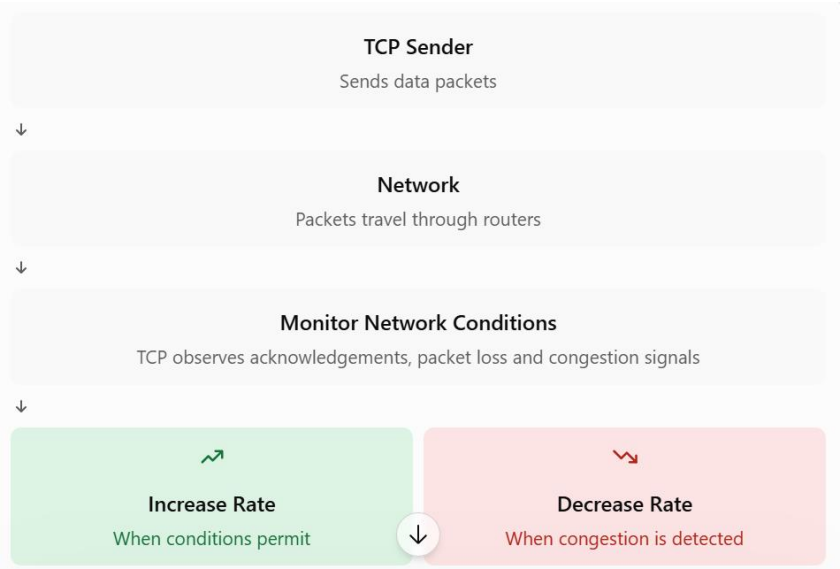
TCP Congestion Control is a mechanism used to control the rate of data transmission to prevent network overload, reduce packet loss, and improve network performance.

Need for Congestion Control

Congestion control is required to:

1. Prevent network overload.
2. Reduce packet loss.
3. Avoid excessive network delay.
4. Improve network performance.
5.
6. Utilize available bandwidth efficiently.
7. Prevent unnecessary retransmissions.

Working of TCP Congestion Control



Congestion Control Techniques

1.Traffic Shaping:

Traffic shaping controls the rate at which packets are sent into the network.

Traffic shaping = Sending packets at a controlled and smooth rate.

It prevents sudden bursts of packets from entering the network.

Example

Suppose an application generates:

100 packets suddenly

Traffic shaping may send them gradually instead of sending all 100 at once.

```
Without shaping:

████████████████████
Packets sent suddenly


With shaping:

██  ██  ██  ██  ██
Packets sent smoothly
```

Advantages

- Reduces congestion
- Controls traffic bursts
- Improves network performance
- Provides smoother traffic flow

Traffic Shapping Algorithms:

1. **Leaky Bucket**
2. **Token Bucket**

2.Traffic Policing:

Traffic policing monitors traffic and checks whether it follows the allowed rate.

Traffic policing = Checking whether network traffic follows the specified limit.

If traffic exceeds the allowed limit, packets may be:

- Dropped
- Delayed
- Marked with lower priority

Example

Suppose a network allows:

10 Mbps

If a user sends:

15 Mbps

the extra traffic may be dropped or marked.

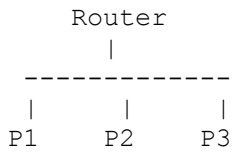
3. Packet Scheduling:

Packet scheduling is the process of deciding **which packet should be transmitted first** when multiple packets are waiting in a router.

Packet scheduling = Deciding the order in which packets are sent.

Example:

Suppose a router has three packets:



The router uses a scheduling method to decide whether:

P1 → P2 → P3

or

P2 → P3 → P1

should be transmitted first.

Common scheduling methods:

- **FIFO (First In First Out):** First packet to arrive is sent first.
- **Priority Scheduling:** High-priority packets are sent first.
- **Round Robin:** Packets/classes are served one after another in turns.
- **Weighted Fair Queuing:** More important traffic can receive a larger share of bandwidth.

Example:

```
Voice packet     → High priority → Sent first
Video packet     → Medium priority
File packet      → Lower priority
```

Advantages:

- Reduces delay for important traffic.
- Provides better QoS.
- Uses bandwidth efficiently.

4.Admission Control:

Admission control decides whether a **new traffic flow or connection should be allowed into the network**.

Admission control = Check the network capacity before accepting new traffic.

Before accepting a new connection, the network checks resources such as:

- Bandwidth
- Buffer space
- Processing capacity
- Delay requirements

Example:

Suppose a network can support **10 video calls**.

```
Network capacity = 10 calls

Existing calls = 10

New call → ✗ Not admitted
```

If only 7 calls are active:

```
Existing calls = 7
Available capacity = 3

New call → ✓ Admitted
```

Advantages:

- Prevents network overload.
- Maintains QoS.
- Ensures sufficient resources for existing connections.

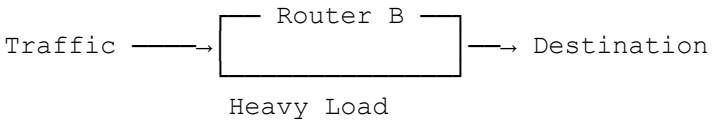
5.Load Balancing:

Load balancing distributes network traffic across **multiple paths, links, or routers**.

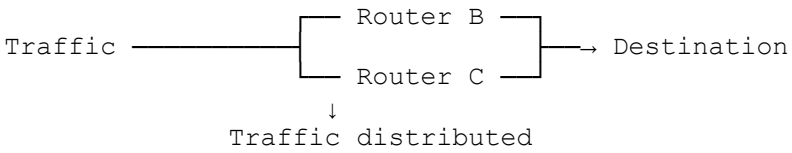
Load balancing = Distributing traffic so that one router or path does not become overloaded.

Example:

Without load balancing:



With load balancing:



For example, if 100 packets need to be sent:

- Path 1 → 50 packets
- Path 2 → 50 packets

Instead of sending all 100 packets through one path.

Advantages:

- Prevents overload.
- Improves throughput.
- Reduces congestion.
- Makes better use of available paths.

15. Desirable bandwidth allocation

Bandwidth is the maximum amount of data that can be transmitted through a network connection in a given time.

Example:

```
Network capacity = 100 Mbps
```

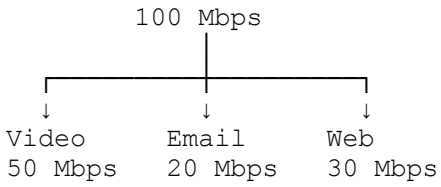
This means the network can carry up to about **100 megabits of data per second** under ideal conditions.

Bandwidth allocation

Bandwidth allocation means **dividing the available network bandwidth among different users, applications, or connections.**

Example

Suppose a network has **100 Mbps** available:



The network allocates bandwidth to different applications.

The goal is to:

- Avoid wasting bandwidth
- Prevent one application from taking all the bandwidth
- Give important applications enough bandwidth
- Maintain good network performance
- Reduce congestion

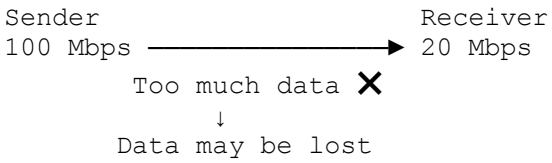
16. Regulating the sending rate

Regulating the sending rate means **controlling how fast a sender sends data into the network**.

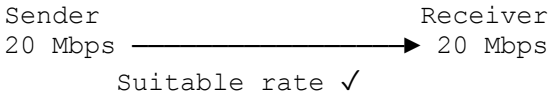
Example

Suppose a sender can send **100 Mbps**, but the receiver can handle only **20 Mbps**.

If the sender continuously sends at 100 Mbps:



So the sending rate should be reduced:



Need of Regulating Sending Rate

Regulating the sending rate helps to:

- Prevent **network congestion**
- Reduce **packet loss**
- Prevent overwhelming the receiver
- Improve network performance
- Use bandwidth efficiently

17. Wireless issues

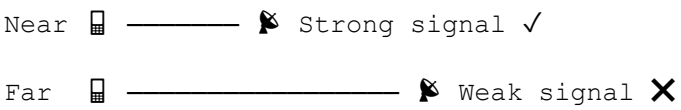
Wireless issues are the problems that occur when data is transmitted through a **wireless network** instead of a wired connection.

In wireless communication, devices communicate using **radio signals**, so the communication can be affected by distance, obstacles, interference, and movement.

Main Wireless Issues

1. Signal Attenuation

The wireless signal becomes **weaker as the distance increases**.



2. Interference

Other devices or signals can disturb wireless communication.

Examples:

- Wi-Fi networks
- Bluetooth devices
- Microwave ovens
- Other radio signals

3. Mobility

Wireless devices can move from one location to another.

Movement can cause:

- Signal strength changes
- Connection interruption
- Handover between access points

4. Security Problems

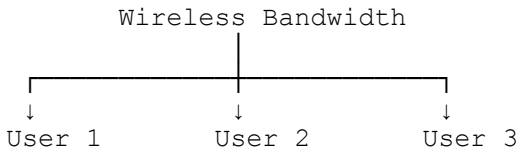
Wireless signals travel through the air, so unauthorized users may try to access or intercept them.

Therefore, wireless networks need:

- Authentication
- Encryption
- Access control

5. Limited Bandwidth

Wireless networks usually have limited shared bandwidth.



Many users sharing the same bandwidth can reduce performance.

6. Variable Connection Quality

Wireless communication quality can change frequently.

18. Internet Transport Protocols

Internet Transport Protocols are protocols used at the **Transport Layer** to provide communication between applications running on different computers over the Internet.

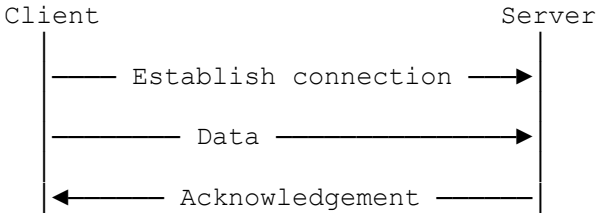
The two main Internet transport protocols are:

- UDP
- TCP

1. TCP – Transmission Control Protocol

TCP provides **reliable and connection-oriented communication**.

Before sending data, TCP establishes a connection between the sender and receiver.



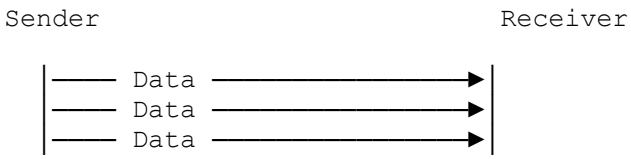
Main features

- Connection-oriented
- Reliable delivery
- Data arrives in order
- Error control
- Flow control
- Congestion control
- Retransmits lost data

2. UDP – User Datagram Protocol

UDP provides **connectionless communication**.

It sends data without first establishing a connection.



UDP does not guarantee that every packet will arrive.

Main features

- Connectionless
- Faster
- Low overhead
- No guaranteed delivery
- No guaranteed ordering
- Suitable for real-time applications

19. Difference between TCP and UDP

TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection-oriented	Connectionless
Reliable communication	No guaranteed delivery
Data is delivered in order	Data order is not guaranteed
Uses acknowledgements	No delivery acknowledgements
Retransmits lost data	Does not normally retransmit lost data
More overhead	Less overhead
Generally slower	Generally faster
Provides flow control	No built-in flow control
Provides congestion control	No built-in congestion control
Header size is 20–60 bytes	Header size is 8 bytes
Provides byte-stream service	Provides message-oriented service
Used in web browsing, email, and file transfer	Used in gaming, DNS, and live streaming

20. Remote Procedure Call (RPC)

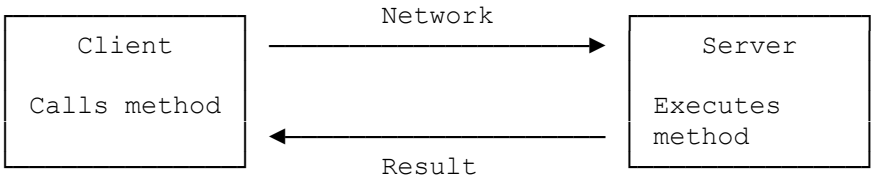
RPC stands for Remote Procedure Call.

It is a technique that allows a program to **call a function/procedure running on another computer as if it were a local function**.

Working of RPC

There are usually two important components:

- **Client**
- **Server**



Steps

Step 1: Client calls a procedure.

```
add(10,20)
```

Step 2: RPC system converts the request into a message.

Step 3: Message is sent through the network.

Step 4: Server receives the request.

Step 5: Server executes the procedure.

```
10 + 20 = 30
```

Step 6: Server sends the result back.

Step 7: Client receives the result.

```
Result = 30
```

21. Real time Transport Protocol(RTP)

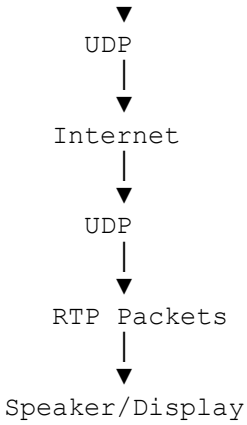
RTP stands for Real-Time Transport Protocol.

It is a **Transport Layer protocol used to deliver audio and video data in real time** over IP networks.

Working of RTP

Suppose you are making a video call:

```
Camera/Microphone
       |
       ▼
   RTP Packets
       |
```



RTP divides the audio/video stream into packets and sends them to the receiver.

Important Features of RTP

1. Sequence Number

Each RTP packet has a **sequence number**.

It helps the receiver identify:

- missing packets
- packet order

2. Timestamp

RTP contains a **timestamp**.

It helps the receiver play audio/video at the correct time.

3. Payload Type

It identifies the type of media being carried, such as:

- Audio
- Video
- Different codecs

4. Real-Time Delivery

RTP is designed for **low-delay communication**.

5. Works with UDP

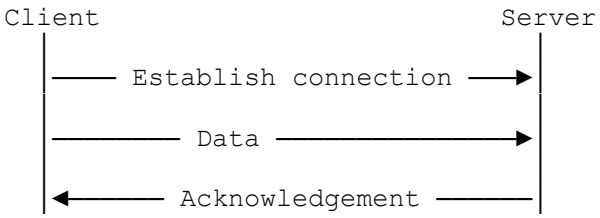
RTP normally uses **UDP** because UDP has less overhead and does not wait for retransmission.

22. Introduction to TCP

TCP stands for Transmission Control Protocol.

TCP is a **Transport Layer protocol** used to provide **reliable communication between applications** over a network.

Before sending data, TCP establishes a connection between the sender and receiver.



Main Features of TCP

1. Connection-Oriented

Before sending data, TCP **establishes a connection** between sender and receiver.



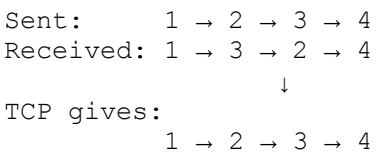
2. Reliable

TCP makes sure that data is delivered reliably.

If a packet is lost, TCP can **retransmit** it.

3. Ordered Delivery

If packets arrive in the wrong order, TCP rearranges them before giving the data to the application.



4. Error Control

TCP detects errors and handles lost or damaged data using mechanisms such as acknowledgements and retransmission.

5. Flow Control

TCP prevents a fast sender from overwhelming a slow receiver.

6. Congestion Control

TCP controls the sending rate when the network becomes congested.

7. Full-Duplex Communication

Both sides can send and receive data at the same time.

23. TCP Service model

The **TCP service model** describes the type of communication service that TCP provides to applications.

Main Features of TCP Service Model

1. Connection-Oriented

Before sending data, TCP establishes a connection between the sender and receiver.

```
Client ──────► Establish Connection ──────► Server
```

After communication is complete, the connection is terminated.

2. Reliable Service

TCP ensures that data reaches the destination reliably.

If data is lost, TCP can **retransmit** it.

```
Sender ── Packet 1 ──► Receiver
Sender ── Packet 2 ──► ✕ Lost
Sender ◄──── ACK ──── Receiver
Sender ── Packet 2 ──► Receiver
```

3. Byte-Stream Service

TCP treats the application data as a **continuous stream of bytes**.

For example:

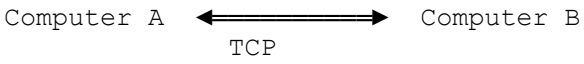
```
Application Data:
HELLO WORLD
```

TCP sends it as a stream of bytes rather than treating each application message as a separate message.

TCP does not preserve message boundaries.

4. Full-Duplex Communication

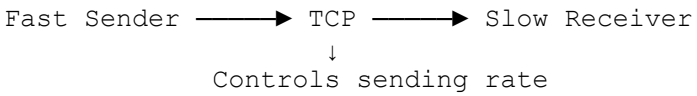
Both sides can send and receive data at the same time.



For example, during a video call, both people can speak and receive audio simultaneously.

5. Flow Control

TCP prevents a fast sender from sending more data than the receiver can handle.



6. Congestion Control

TCP also controls the sending rate when the **network is congested**.

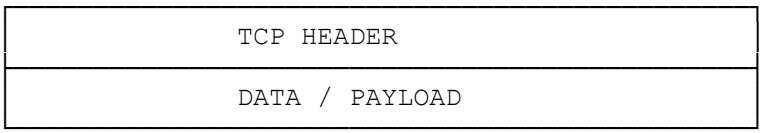
This helps reduce packet loss and network overload.

7. Error Control

TCP detects problems such as lost or corrupted data and uses acknowledgements and retransmission to provide reliable communication.

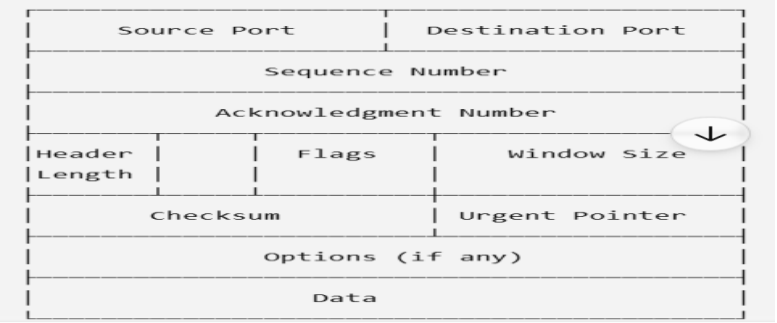
24. TCP segment Header

A **TCP segment** consists of a **TCP header + data**.



The TCP header contains important information needed for **reliable communication**.

TCP Header Format



Important Fields

1. Source Port

Identifies the **application sending the data**.

Example:

```
Source Port = 5000
```

2. Destination Port

Identifies the **application receiving the data**.

Example:

```
Destination Port = 5000
```

3. Sequence Number

Identifies the **position of data bytes** in the TCP stream.

It helps TCP:

- arrange data in the correct order
- detect missing data

4. Acknowledgment Number

Tells the sender **which data has been successfully received** and what byte is expected next.

5. Header Length

Indicates the **size of the TCP header**.

6. Flags

Contains control information related to fragmentation.

7. Window Size

Used for **flow control**.

It tells the sender how much data the receiver can accept.

8. Checksum

Used to detect **errors or corruption** in the TCP segment.

9. Urgent Pointer

Indicates the position of **urgent data** when the URG flag is set.

10. Options

Provides additional TCP features, such as maximum segment size and other parameters.

11. Data

Contains the **actual application information**.

25. 7. Window Size

Used for **flow control**.

It tells the sender how much data the receiver can accept.

8. Checksum

Used to detect **errors or corruption** in the TCP segment.

9. Urgent Pointer

Indicates the position of **urgent data** when the URG flag is set.

10. Options

Provides additional TCP features, such as maximum segment size and other parameters.

11. Data

Contains the **actual application information**.

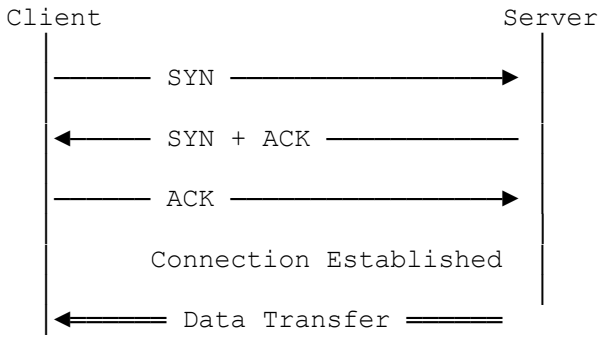
25. TCP Connection establishment

TCP is a **connection-oriented protocol**, so before sending actual data, it establishes a connection between the client and server.

TCP uses a process called the **Three-Way Handshake**.

Three-Way Handshake = SYN → SYN-ACK → ACK

Three-Way Handshake



Step 1: SYN

The **client sends a SYN segment** to the server.

SYN = Synchronize

It requests:

"Can we establish a TCP connection?"

It also contains the client's initial sequence number.

Step 2: SYN + ACK

The server receives the SYN and sends back **SYN + ACK**.

- **SYN** → Server also wants to establish the connection.
- **ACK** → Server acknowledges the client's SYN.

Meaning:

"I received your request, and I am ready to communicate."

Step 3: ACK

The client sends an **ACK** back to the server.

Meaning:

"I received your response."

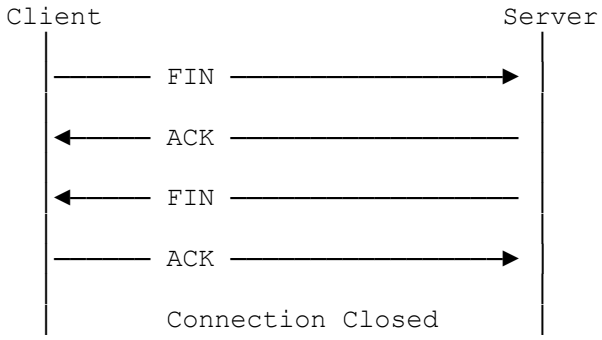
Now the TCP connection is established.

26. TCP connection release

CP connection release is the process of **properly closing a TCP connection after data communication is completed**.

TCP normally uses a **four-way handshake** to terminate a connection.

Four-Way Handshake



Step 1: FIN

The client sends a **FIN** segment to the server.

FIN = Finish

Meaning:

"I have finished sending data."

Step 2: ACK

The server sends an **ACK** to acknowledge the client's FIN.

Meaning:

"I received your request to close."

The server can still send remaining data if necessary.

Step 3: FIN

After the server finishes sending its data, it sends its own **FIN**.

Meaning:

"I have also finished sending data."

Step 4: ACK

The client sends the final **ACK**.

Meaning:

"I received your FIN."

The TCP connection is then closed.